# Soft Computing Lab Manual: Theory, Code and Output

Python code (NumPy only) with real outputs for Experiments 1 to 10. Run each script with `python3 expN.py`.

## Exp 1: Case Study on an Application of Soft Computing

**Aim:** To study an application of soft computing (fuzzy tip advisor / fuzzy appliance control).

**Theory:** Soft computing tolerates imprecision, uncertainty and approximation. Its components are Neural Networks, Fuzzy Logic and Genetic Algorithms. In fuzzy applications like washing machines, ACs or medical diagnosis, vague inputs ("dirty", "hot") are mapped to membership degrees, rules are applied, and a crisp output is produced.

**Case:** Service quality (0 to 10) decides the tip %. Fuzzy sets: Poor, Good, Great (triangular). Output tips: Poor = 5%, Good = 15%, Great = 25%. Defuzzification by weighted average.

```
def tri(x,a,b,c):
    if x<=a or x>=c: return 0.0
    return (x-a)/(b-a) if x<=b else (c-x)/(c-b)
def tip(service):
    poor=tri(service,-1,0,5); good=tri(service,0,5,10);
great=tri(service,5,10,11)
    num=poor*5+good*15+great*25; den=poor+good+great
    return (poor,good,great), num/den
for s in [2,5,7,9]:
    m,t=tip(s)
    print(f"service={s} memberships(poor,good,great)={tuple(round(v,2) for v
in m)} -> tip={t:.2f}%")
```

**Output:**

```
service=2 memberships(poor,good,great)=(0.6, 0.4, 0.0) -> tip=9.00%
service=5 memberships(poor,good,great)=(0.0, 1.0, 0.0) -> tip=15.00%
service=7 memberships(poor,good,great)=(0.0, 0.6, 0.4) -> tip=19.00%
service=9 memberships(poor,good,great)=(0.0, 0.2, 0.8) -> tip=23.00%
```

**Conclusion:** Fuzzy logic gives smooth, human-like output instead of abrupt jumps.

# Exp 2: McCulloch-Pitts Model (AND, OR, XOR)

**Aim:** To implement the M-P neuron for AND, OR and XOR.

**Theory:** The M-P neuron is a binary threshold unit: y = 1 if $\Sigma w_i x_i \geq \theta$, else 0. Weights are fixed (no learning). Inhibitory inputs have negative weights.

- AND: w = (1, 1), θ = 2
- OR: w = (1, 1), θ = 1
- XOR is not linearly separable, so two layers are used: $z_1 = x_1$ AND NOT $x_2$ (w = 2, −1; θ = 2), $z_2$ = NOT $x_1$ AND $x_2$ (w = −1, 2; θ = 2), y = $z_1$ OR $z_2$.

```
def mp(x,w,th): return int(sum(a*b for a,b in zip(x,w))>=th)
ins=[(0,0),(0,1),(1,0),(1,1)]
print("x1 x2 | AND OR XOR")
for x in ins:
    a=mp(x,(1,1),2); o=mp(x,(1,1),1)
    z1=mp(x,(2,-1),2); z2=mp(x,(-1,2),2); xr=mp((z1,z2),(1,1),1)
    print(f"{x[0]}  {x[1]}  |  {a}   {o}   {xr}")
```

**Output:**

```
x1 x2 | AND OR XOR
0  0  |  0   0   0
0  1  |  0   1   1
1  0  |  0   1   1
1  1  |  1   1   0
```

**Conclusion:** AND and OR work with one neuron. XOR needs a hidden layer.

# Exp 3: Supervised Learning (Perceptron Rule)

**Aim:** To train a neural network with a supervised learning rule to classify the AND function.

**Theory:** Supervised learning uses input-target pairs. Perceptron rule: w_new = w_old + η(t − y)x and b_new = b_old + η(t − y). The output is y = 1 if w·x + b ⩾ 0, else 0.

**Algorithm:**

1. Initialize w = 0, b = 0, choose η.
2. For each pattern, compute y.
3. Update weights using the error (t − y).
4. Repeat epochs until the total error is 0.

```
import numpy as np
X=np.array([[0,0],[0,1],[1,0],[1,1]]); t=np.array([0,0,0,1])
w=np.zeros(2); b=0.0; eta=0.1
for ep in range(1,20):
    err=0
    for x,tg in zip(X,t):
        y=1 if w@x+b>=0 else 0
        e=tg-y; w=w+eta*e*x; b=b+eta*e; err+=abs(e)
    print(f"epoch {ep}: w={np.round(w,2)}, b={b:.2f}, errors={err}")
    if err==0: break
print("Predictions:",[1 if w@x+b>=0 else 0 for x in X])
```

**Output:**

```
epoch 1: w=[0.1 0.1], b=0.00, errors=2
epoch 2: w=[0.2 0.1], b=-0.10, errors=3
epoch 3: w=[0.2 0.1], b=-0.20, errors=3
epoch 4: w=[0.2 0.1], b=-0.20, errors=0
Predictions: [0, 0, 0, 1]
```

**Conclusion:** The perceptron converged in 4 epochs because AND is linearly separable (Perceptron Convergence Theorem).

## Exp 4: Unsupervised Learning (Competitive Learning and Hebb)

**Aim:** To cluster input vectors using winner-take-all learning, and to demonstrate the Hebb rule.

**Theory:** No targets are given. In competitive (Kohonen) learning, the neuron whose weight vector is closest to the input (minimum Euclidean distance) wins, and only the winner updates: w_j = w_j + α(x − w_j). α decreases with time. The Hebb rule is Δw = η·x·y.

```
import numpy as np
X=np.array([[1,1],[1.5,2],[1,1.8],[8,8],[9,9.5],[8.5,7.5]])
W=np.array([[1.0,1.0],[9.0,9.0]]); alpha=0.5
for ep in range(1,4):
    for x in X:
        j=np.argmin(((W-x)**2).sum(1)); W[j]+=alpha*(x-W[j])
    alpha*=0.5
    print(f"epoch {ep}: weights={np.round(W,2).tolist()}")
print("Clusters:",[int(np.argmin(((W-x)**2).sum(1))) for x in X])
# Hebb rule
x=np.array([1,-1,1]); w=np.zeros(3); y=1
w=w+1*x*y; print("Hebb weights after 1 pattern:",w)
```

**Output:**

```
epoch 1: weights=[[1.12, 1.65], [8.62, 8.25]]
epoch 2: weights=[[1.15, 1.66], [8.58, 8.26]]
epoch 3: weights=[[1.15, 1.65], [8.56, 8.28]]
Clusters: [0, 0, 0, 1, 1, 1]
Hebb weights after 1 pattern: [ 1. -1.  1.]
```

**Conclusion:** The network grouped the six points into two clusters without any target labels.

# Exp 5: Fuzzy Operations and Relations

**Aim:** To implement fuzzy set operations and max-min composition of relations.

**Theory:** A fuzzy set assigns $\mu(x) \in [0, 1]$.

- Union: max(μA, μB)
- Intersection: min(μA, μB)
- Complement: 1 − μA
- Algebraic sum: a + b − ab
- Algebraic product: ab
- Max-min composition: μ(x,z) = max over y of min(μR(x,y), μS(y,z))

```
import numpy as np
A=np.array([0.2,0.5,0.8,1.0]); B=np.array([0.9,0.4,0.6,0.1])
print("A =",A); print("B =",B)
print("Union      :",np.maximum(A,B)); print("Intersect  :",np.minimum(A,B))
print("Complement A:",np.round(1-A,2))
print("Alg. sum   :",np.round(A+B-A*B,2)); print("Alg.
product:",np.round(A*B,2))
R=np.array([[0.3,0.7],[0.8,0.2],[0.5,0.9]]); S=np.array([[0.6,0.4,0.9],
[0.5,0.8,0.1]])
C=np.array([[max(min(R[i,k],S[k,j]) for k in range(2)) for j in range(3)] for
i in range(3)])
print("Max-min composition R o S:\n",C)
```

**Output:**

```
A = [0.2 0.5 0.8 1. ]
B = [0.9 0.4 0.6 0.1]
Union      : [0.9 0.5 0.8 1. ]
Intersect  : [0.2 0.4 0.6 0.1]
Complement A: [0.8 0.5 0.2 0. ]
Alg. sum   : [0.92 0.7  0.92 1.  ]
Alg. product: [0.18 0.2  0.48 0.1 ]
Max-min composition R o S:
 [[0.5 0.7 0.3]
 [0.6 0.4 0.8]
 [0.5 0.8 0.5]]
```

**Conclusion:** All standard fuzzy operations and the max-min composition were verified.

# Exp 6: Defuzzification Methods

**Aim:** To implement common defuzzification methods.

**Theory:** Defuzzification converts a fuzzy output set into one crisp value.

- Centroid (COG): $z^* = \Sigma\mu(z) \cdot z / \Sigma\mu(z)$
- Bisector: the z that splits the area into two equal halves
- Mean / First / Last of maxima: the average, first and last z where μ is maximum
- Weighted average: $\Sigma\mu_i z_i / \Sigma\mu_i$ (using peak values)
- λ-cut: elements with $\mu \geq \lambda$

```
import numpy as np
z=np.arange(0,101,1.0)
def tri(x,a,b,c): return np.maximum(np.minimum((x-a)/(b-a),(c-x)/(c-b)),0)
mu=np.maximum(np.minimum(tri(z,0,25,50),0.4),np.minimum(tri(z,25,60,95),0.7))
cen=(mu*z).sum()/mu.sum()
cs=np.cumsum(mu); bis=z[np.searchsorted(cs,cs[-1]/2)]
mx=z[mu==mu.max()]
print(f"Centroid (COG)   = {cen:.2f}")
print(f"Bisector         = {bis:.2f}")
print(f"Mean of maxima   = {mx.mean():.2f}")
print(f"First of maxima  = {mx[0]:.2f}")
print(f"Last of maxima   = {mx[-1]:.2f}")
print(f"Weighted average (peaks 25,60 with 0.4,0.7) =
{(0.4*25+0.7*60)/(1.1):.2f}")
lam=0.5; A=np.array([0.2,0.6,0.9,0.4]); print("Lambda-cut(0.5) of",A,"->",
(A>=lam).astype(int))
```

**Output:**

```
Centroid (COG)   = 49.53
Bisector         = 52.00
Mean of maxima   = 60.00
First of maxima  = 50.00
Last of maxima   = 70.00
Weighted average (peaks 25,60 with 0.4,0.7) = 47.27
Lambda-cut(0.5) of [0.2 0.6 0.9 0.4] -> [0 1 1 0]
```

**Conclusion:** Different methods give different crisp values for the same fuzzy set. Centroid is the most widely used.

## Exp 7: Fuzzy Controller (Washing Machine)

**Aim:** To design a Mamdani fuzzy controller for a washing machine.

**Theory:** An FLC has four blocks: Fuzzifier → Rule base + Inference engine → Defuzzifier.

- Inputs: Dirt and Grease (0 to 100), each Low / Medium / High (triangular)
- Output: Wash time (0 to 60 min), Short / Medium / Long
- Inference: min for AND, max for aggregation, centroid defuzzification

**Rule table (rows = dirt, columns = grease):**

| Dirt \ Grease | Low | Medium | High |
|---|---|---|---|
| Low | Short | Short | Medium |
| Medium | Short | Medium | Long |
| High | Medium | Long | Long |

```
import numpy as np
def tri(x,a,b,c): return np.maximum(np.minimum((x-a)/(b-a),(c-x)/(c-b)),0)
lvl={'L':(-50,0,50),'M':(0,50,100),'H':(50,100,150)}
tm={'S':(-30,0,30),'M':(10,30,50),'L':(30,60,90)}
rules={('L','L'):'S',('L','M'):'S',('L','H'):'M',('M','L'):'S',('M','M'):'M',
       ('M','H'):'L',('H','L'):'M',('H','M'):'L',('H','H'):'L'}
dirt,grease=60,70
z=np.arange(0,61,0.5); agg=np.zeros_like(z)
for (d,g),o in rules.items():
    f=min(tri(dirt,*lvl[d]),tri(grease,*lvl[g]))
    if f>0:
        print(f"Rule dirt={d},grease={g} -> time={o}  firing={f:.2f}")
        agg=np.maximum(agg,np.minimum(f,tri(z,*tm[o])))
print(f"\nInputs: dirt={dirt}, grease={grease}")
print(f"Wash time (centroid) = {(agg*z).sum()/agg.sum():.2f} min")
```

**Output:**

```
Rule dirt=M,grease=M -> time=M  firing=0.60
Rule dirt=M,grease=H -> time=L  firing=0.40
Rule dirt=H,grease=M -> time=L  firing=0.20
Rule dirt=H,grease=H -> time=L  firing=0.20

Inputs: dirt=60, grease=70
Wash time (centroid) = 36.03 min
```

**Conclusion:** For dirt 60 and grease 70, the controller recommends about 36 minutes of washing.

# Exp 8: Genetic Algorithm Operators

**Aim:** To apply selection, crossover and mutation to maximize $f(x) = x^2$ for x in 0 to 31.

**Theory:** A GA is a population-based search inspired by natural selection.

1. Encode x as a 5-bit string.
2. Create the initial population (6 individuals).
3. Evaluate fitness $f(x) = x^2$.
4. Select parents (roulette wheel).
5. Single-point crossover (probability 0.8).
6. Bit-flip mutation (probability 0.05).
7. Keep the best individual (elitism) and repeat for each generation.

```
import random
random.seed(1)
f=lambda x:x*x
dec=lambda b:int(''.join(map(str,b)),2)
pop=[[random.randint(0,1) for _ in range(5)] for _ in range(6)]
print("Initial:",[dec(p) for p in pop])
for g in range(1,9):
    fit=[f(dec(p)) for p in pop]; tot=sum(fit)
    def pick():
        r=random.uniform(0,tot); s=0
        for p,ft in zip(pop,fit):
            s+=ft
            if s>=r: return p
        return pop[-1]
    best=pop[fit.index(max(fit))][:]
    new=[best]
    while len(new)<6:
        p1,p2=pick(),pick()
        if random.random()<0.8:
            c=random.randint(1,4); p1,p2=p1[:c]+p2[c:],p2[:c]+p1[c:]
        for ch in (p1,p2):
            ch=[1-b if random.random()<0.05 else b for b in ch]
            if len(new)<6: new.append(ch)
    pop=new
    fit=[f(dec(p)) for p in pop]
    print(f"Gen {g}: pop={[dec(p) for p in pop]} best x=
{dec(pop[fit.index(max(fit))])} f={max(fit)}")
```

**Output:**

```
Initial: [5, 28, 22, 25, 1, 9]
Gen 1: pop=[28, 30, 17, 25, 25, 24] best x=30 f=900
Gen 2: pop=[30, 17, 26, 27, 29, 25] best x=30 f=900
Gen 3: pop=[30, 26, 31, 26, 25, 26] best x=31 f=961
Gen 4: pop=[31, 31, 27, 27, 24, 10] best x=31 f=961
Gen 5: pop=[31, 31, 15, 27, 24, 27] best x=31 f=961
Gen 6: pop=[31, 24, 24, 27, 18, 27] best x=31 f=961
Gen 7: pop=[31, 26, 27, 25, 30, 31] best x=31 f=961
Gen 8: pop=[31, 23, 26, 27, 29, 31] best x=31 f=961
```

**Conclusion:** The GA reached the optimum x = 31 (f = 961) in 3 generations.

# Exp 9: ANFIS Case Study

**Aim:** To build a neuro-fuzzy (ANFIS) model that learns y = sin(x).

**Theory:** ANFIS is a Sugeno fuzzy system with neural-network style learning, in 5 layers.

1. Fuzzification: Gaussian membership $\mu_i(x)$
2. Rule firing strength $w_i = \mu_i(x)$
3. Normalization: $\bar{w}_i = w_i / \Sigma w$
4. Consequent: $\bar{w}_i(p_i x + r_i)$
5. Output: sum of layer 4

**Hybrid learning:** in the forward pass, consequent parameters (p, r) are found by least squares (LSE). In the backward pass, premise parameters (centers, sigmas) are updated by gradient descent (here the gradient is computed numerically for simplicity).

```
import numpy as np
x=np.linspace(0,2*np.pi,50); y=np.sin(x)
c=np.array([1.0,3.1,5.2]); s=np.array([1.0,1.0,1.0])
def fwd(c,s):
    mu=np.exp(-((x[:,None]-c)**2)/(2*s**2))           # layer 1
    w=mu/mu.sum(1,keepdims=True)                       # layers 2-3
    A=np.hstack([w*x[:,None],w])                       # layer 4 design
matrix
    th=np.linalg.lstsq(A,y,rcond=None)[0]              # LSE (forward pass)
    return A@th,th
def rmse(c,s): return np.sqrt(np.mean((fwd(c,s)[0]-y)**2))
lr=0.05
for ep in range(1,41):
    e=rmse(c,s); g=np.zeros(6)
    for i in range(3):                                 # gradient (backward
pass)
        d=1e-4; cc=c.copy(); cc[i]+=d; g[i]=(rmse(cc,s)-e)/d
        ss=s.copy(); ss[i]+=d; g[3+i]=(rmse(c,ss)-e)/d
    c-=lr*g[:3]; s-=lr*g[3:]
    if ep in(1,10,20,30,40): print(f"epoch {ep:2d}: RMSE={rmse(c,s):.4f}")
print("Final centers:",np.round(c,2),"sigmas:",np.round(s,2))
p,_=fwd(c,s)
for i in (0,12,25,37,49): print(f"x={x[i]:.2f} actual={y[i]:.3f} ANFIS=
{p[i]:.3f}")
```

**Output:**

```
epoch  1: RMSE=0.0263
epoch 10: RMSE=0.0225
epoch 20: RMSE=0.0189
epoch 30: RMSE=0.0159
epoch 40: RMSE=0.0133
Final centers: [1.03 3.11 5.17] sigmas: [1.05 1.13 1.07]
x=0.00 actual=0.000 ANFIS=0.019
x=1.54 actual=0.999 ANFIS=1.015
x=3.21 actual=-0.064 ANFIS=-0.058
x=4.74 actual=-0.999 ANFIS=-1.014
x=6.28 actual=-0.000 ANFIS=-0.020
```

**Conclusion:** The error falls steadily as the membership functions are tuned, and the ANFIS output closely follows sin(x).

## Exp 10: Mini Project (Extractive Text Summarizer)

**Aim:** To build a text summarizer that selects the most important sentences.

**Theory:** Extractive summarization scores each sentence and keeps the top ones in original order.

1. Split text into sentences and remove stop words.
2. Count word frequencies.
3. Score each sentence = average frequency of its words.
4. Pick the top-N sentences.

**Soft computing link:** the sentence score can be fuzzified (e.g. position, length, keyword score → Low/Medium/High) and combined with fuzzy rules, or the feature weights can be tuned with a GA.

```
import re
from collections import Counter
text="""Soft computing deals with imprecision and uncertainty. Neural
networks learn patterns from data. Fuzzy logic handles vague information
using membership degrees. Genetic algorithms search for optimal solutions
using selection and mutation. Hybrid systems combine neural networks, fuzzy
logic and genetic algorithms. The weather was pleasant yesterday. Soft
computing is used in controllers, medical diagnosis and text mining."""
stop=
{'the','and','is','in','was','for','from','with','of','a','to','using','used'
,'deals'}
sents=re.split(r'(?<=\.)\s+',text.strip())
words=lambda s:[w for w in re.findall(r'\w+',s.lower()) if w not in stop]
freq=Counter(w for s in sents for w in words(s))
score=[sum(freq[w] for w in words(s))/len(words(s)) for s in sents]
print("Sentence scores (avg word frequency):")
for i,s in enumerate(score): print(f" S{i+1}: {s:.2f}")
top=sorted(sorted(range(len(sents)),key=lambda i:-score[i])[:3])
print("\nSUMMARY:"); print(" ".join(sents[i] for i in top))
```

**Output:**

```
Sentence scores (avg word frequency):
 S1: 1.50
 S2: 1.40
 S3: 1.29
 S4: 1.29
 S5: 1.67
 S6: 1.00
 S7: 1.29

SUMMARY:
Soft computing deals with imprecision and uncertainty. Neural networks learn
patterns from data. Hybrid systems combine neural networks, fuzzy logic and
genetic algorithms.
```

**Conclusion:** The off-topic weather sentence got the lowest score (1.00) and was dropped from the summary.

**Report headings for the write-up:** problem statement, objectives, block diagram, algorithm, dataset, tools (Python), results, limitations, future scope (abstractive models, fuzzy scoring).